hashlock.

Security Audit

OpenZK 8th (DeFi)

June 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

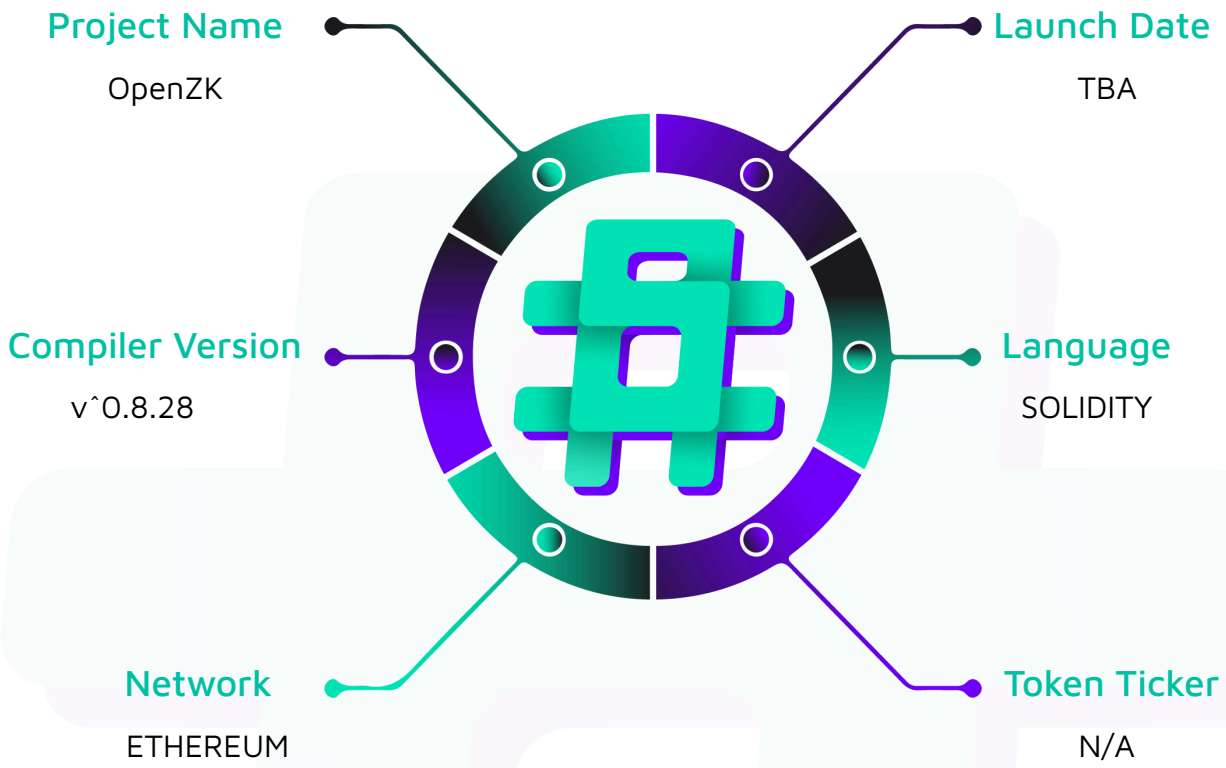
The OpenZK team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

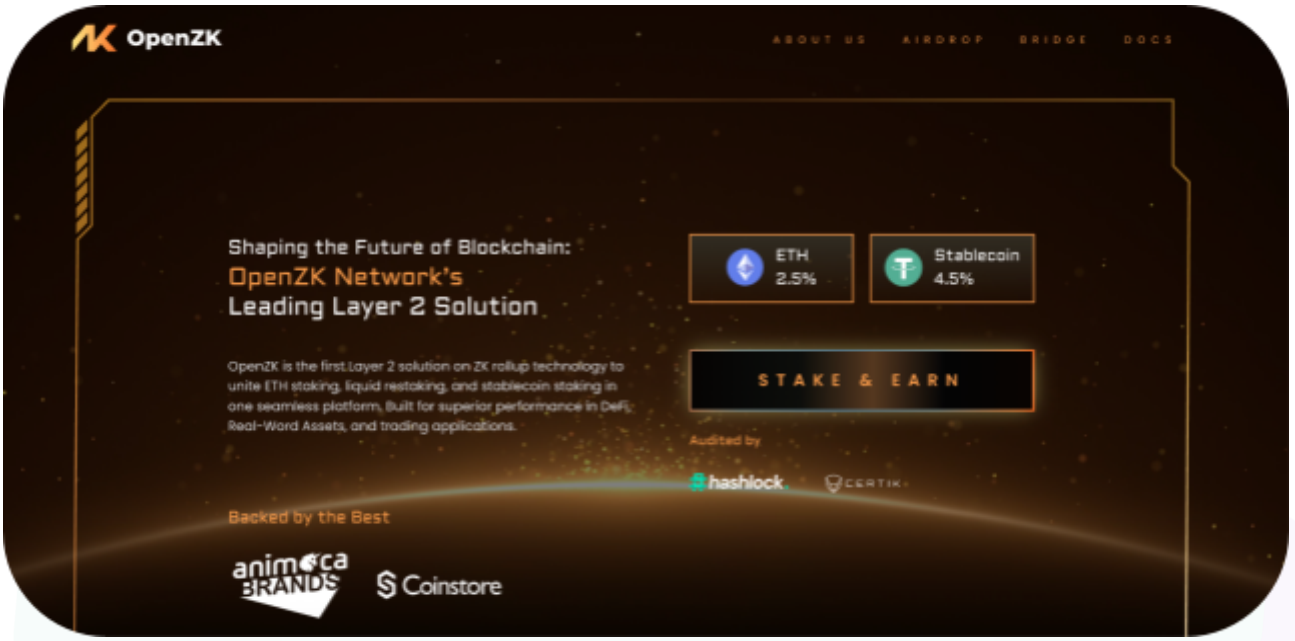
OpenZK is an innovative Layer 2 (L2) network project that leverages Zero-Knowledge (ZK) Rollup technology to enhance Ethereum's scalability and transaction efficiency while maintaining high security. By integrating native ETH staking and restaking capabilities, OpenZK not only bolsters scalability but also creates new revenue streams for users. The platform's leadership includes co-founder Dave Sandor, formerly an Executive Director at Goldman Sachs Asia-Pacific, co-founder and CTO Lucas Cullen, an early contributor to Ethereum's development, and co-founder Jenna Wayne. OpenZK distinguishes itself with a Dual Gas Fee Mechanism, allowing users to pay gas fees with its native and protocol tokens, enhancing network flexibility and creating sustained demand for its protocol token.

Project Name: OpenZK
Project Type: DeFi
Compiler Version: ^0.8.28
Website: https://www.openzk.net/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the OpenZK project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**OpenZK Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**June, 2025**
Contract 1	Staking.sol
Contract 1 MD5 Hash	9fec3922b9a70bb0d8771f2e90b692a6
Audited GitHub Commit Hash	ea2d6bb9ea0fa214411b9428f73474e4ce384adb
Fix Review GitHub Commit Hash	b2ebebec9543e8fb3839083f0e271fbbef241f95

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section, and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Medium severity vulnerabilities

1 Low severity vulnerability

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Staking.sol - Allows users to: - Deposit OZK tokens for staking with epoch-based time-weighted rewards - Check in to specific epochs to participate in reward distribution and bridge tokens to L2 - Queue withdrawal requests with a configurable delay period - Process queued withdrawals after the delay period expires - View their staking balances, effective balances per epoch, and check-in history - Allows admins to: - Set the withdrawal delay period for security - Adjust the check-in fee required for epoch participation	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the OpenZK project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the OpenZK project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Staking#setWithdrawDelay()&setCheckInFee() - Lack of Upper Bounds on Critical Parameters Set by Moderator

Description

The contract allows an address with the `MODERATOR_ROLE` to set the withdrawal delay and the check-in fee. However, the functions `setWithdrawDelay()` and `setCheckInFee()` do not enforce any upper bounds on these values, allowing a moderator to set them to arbitrarily large numbers.

Vulnerability Details

The `setWithdrawDelay()` and `setCheckInFee()` functions can be called by a `MODERATOR_ROLE` to update their respective parameters:

```
function setWithdrawDelay(uint256 newDelay) external onlyRole(MODERATOR_ROLE) {
        require(newDelay != withdrawDelay, "Staking: new delay must be different");
        withdrawDelay = newDelay;
        emit WithdrawDelaySet(newDelay);
    }

function setCheckInFee(uint256 newCheckInFee) external onlyRole(MODERATOR_ROLE) {
        require(newCheckInFee != checkInFee, "Staking: new fee must be different");
        checkInFee = newCheckInFee;
        emit CheckInFeeSet(newCheckInFee);
    }
```

The issue is that neither function validates the new value against a maximum limit. A malicious or compromised moderator could therefore call `setWithdrawDelay()` with an extremely high value, effectively locking user funds for an unreasonable period of time. Similarly, the moderator could call `setCheckInFee()` with a prohibitively expensive fee, making it economically impossible for users to check in for an epoch. This action would prevent all stakers from making their deposits eligible for rewards, thus breaking the core of the staking system.

Impact

Setting an extreme withdrawal delay can indefinitely trap user funds within the contract, leading to a permanent loss of liquidity for stakers. Setting an extreme check-in fee denies all users their rightful staking rewards, which undermines the fundamental purpose and incentive of the protocol.

Recommendation

It is recommended to introduce maximum values for both the withdrawal delay and the check-in fee.

Status

Resolved

[M-02] Staking#checkIn() - DOS in checkIn Function due to Shared Gas Fund

Description

The `checkIn` function pays for a user's L2 bridge transaction fee from a shared pool of `gasToken` held by the contract. An attacker can drain this shared pool, causing the `checkIn` function to become unusable for all other stakers.

Vulnerability Details

When a user calls the `checkIn` function, the contract calculates a variable `gasCost` required to execute a cross-chain transaction on the bridge.

```
function checkIn() external nonReentrant {

        require(userHasDeposits(msg.sender), "Staking: no deposits");



        uint128 epochId = getCurrentEpoch();

        require(epochId > 0, "Staking: Cannot checkIn epoch 0");

            require(!hasUserCheckInEpoch(msg.sender, epochId), "Staking: already checked
in");



        uint256 _l2GasLimit = 500_000;

        uint256 _l2GasPerPubdataByteLimit = 800;



                    uint256 gasCost = l2TransactionBaseCost(tx.gasprice, _l2GasLimit,
_l2GasPerPubdataByteLimit);

           require(gasToken.balanceOf(address(this)) >= gasCost, "Insufficient funds for
bridge");



        IERC20 token = IERC20(stakingToken);

        token.safeTransferFrom(msg.sender, address(this), checkInFee);



        uint256 diff = _updateCheckpoint(epochId, balances[msg.sender], 0, true);

        _updatePoolSize(epochId, diff);



        // bridge

        _bridgeToken(gasCost, _l2GasLimit, _l2GasPerPubdataByteLimit);



        emit CheckIn(msg.sender, epochId);

    }
```

Here, the contract then pays this fee using its `gasToken` balance, which is confirmed by the check requirement (`gasToken.balanceOf(address(this)) >= gasCost, ...`). This `gasToken` balance is a common resource shared among all users. A malicious actor can make minimal deposits across multiple accounts to meet the requirement (`userHasDeposits(msg.sender), ...`) condition. Since each account can call `checkIn` once per epoch, an attacker with multiple accounts can repeatedly drain the contract's `gasToken` reserves by forcing it to pay `gasCost` on behalf of each account within the same epoch. Alternatively, a single attacker could gradually deplete the pool over multiple epochs, though this would be much slower given the epoch duration. Once the balance is insufficient, the `checkIn` function will revert for all legitimate users.

Impact

An attacker can drain the contract's shared gas token pool, causing the `checkIn` function to become unusable for users, preventing them from participating in reward distribution.

Recommendation

Modify the `checkIn` function to require users to pay their own gas costs by having them transfer the required `gasToken` amount to the contract before the bridge operation, instead of using the contract's shared `gasToken` balance.

Status

Resolved

Low

[L-01] Contracts - Missing Minimum Deposit check, Allows Zero Effective Stake

Description

The deposit function lacks a minimum deposit requirement, allowing users to make extremely small deposits that result in zero effective stake for reward calculations. When very small amounts (a few wei) are deposited, especially late in an epoch when the time multiplier is small, the fixed-point arithmetic calculation in `computeNewMultiplier` rounds the effective stake down to zero due to Solidity's integer division. This occurs because the calculation amount * `currentMultiplier` / `BASE_MULTIPLIER` produces a result less than 1, which gets truncated to zero. While the user's actual deposit is still recorded and withdrawable, it contributes nothing to the reward pool, creating an inconsistency between the deposited amount and the reward-earning potential.

Recommendation

Implement a minimum deposit check in the deposit function to ensure all deposits generate a meaningful effective stake for reward calculations.

Status

Resolved

Centralisation

The OpenZK project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the OpenZK project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au